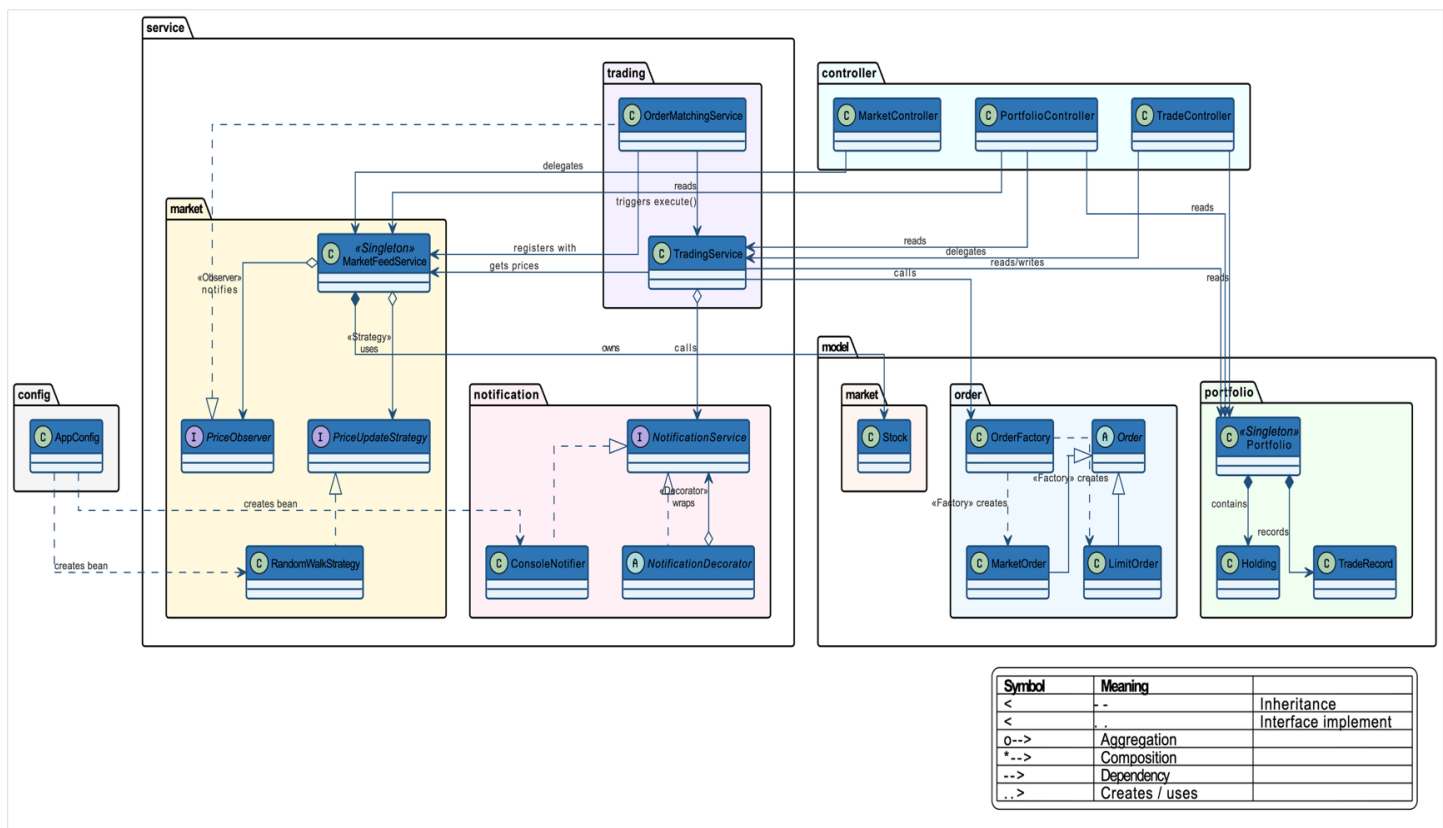


Trade Simulator App Component Diagram



1. Strategy Classes: PriceUpdateStrategy (interface), RandomWalkStrategy (implementation), MarketFeedService (context) Justification: Encapsulates the price calculation algorithm behind an interface so that MarketFeedService can swap pricing without any modification to itself.

2. Observer Classes: PriceObserver (interface), MarketFeedService (subject), OrderMatchingService (observer) Justification: Decouples the market feed from limit-order evaluation so that OrderMatchingService is automatically notified on every price tick without MarketFeedService knowing anything about order matching logic.

3. Decorator Classes: NotificationService (interface), ConsoleNotifier (base), NotificationDecorator (abstract wrapper), EmailDecorator / SmsDecorator (Week 2) Justification: Allows notification channels to be stacked at runtime so that adding email or SMS in Week 2 requires only one new class and one line change in AppConfig, with zero changes to TradingService.

4. Factory Classes: OrderFactory (creator), Order (abstract product), MarketOrder / LimitOrder (concrete products) Justification: Centralises order instantiation so that TradingService never calls new MarketOrder() or new LimitOrder() directly, meaning a new order type (e.g. StopLossOrder) can be added in Week 2 by changing only OrderFactory.

5. Singleton Classes: Portfolio (@Component), MarketFeedService (@Component) Justification: Spring's @Component guarantees a single shared instance of both Portfolio and MarketFeedService so that all controllers and services always read and write the same cash balance, holdings, and live price map.